업데이트 방향 V4.1, V4.2

1) 범위 및 용어

범위

- e2e(Ginkgo)에서 수행하는 SLO/SLI 계측을 **계측 방식 (Trigger/Observe/Evaluate)** 관점으로 재정리하고,
- v3/v4(현재)에서 **지원 중인 조합**과,
- v4.1/v4.2에서 **추가/정교화할 조합** 및 구현 계획을 명시한다.
- 결과 산출물(summary JSON)은 당분간 `schemaVersion="slo.v3"` **유지**를 기본 정책으로 한다.

용어

- **Trigger(경계 생성)**: 측정 구간의 시작/끝(Boundary)을 정하는 방식
  - *No-trigger(계측자 경계)*: 세션이 `Start/End` 시각을 찍음
  - *Trigger-based(시스템 경계)*: annotation/condition/status 등 시스템 신호로 경계를 결정
- **Observe(스냅샷 수집)**: 경계 시점에 입력 스냅샷(메트릭 맵)을 가져오는 방식
  - *External observe*: `/metrics` 등 외부 노출을 스크랩(curl pod 포함)
  - *Internal observe*: 테스트 내부/프로세스 내부에서 직접 스냅샷 구성(프로덕션 침투 금지 원칙 준수 필요)
- **Evaluate(엔진)**: (start/end) 스냅샷으로 SLI 계산/판정/summary 생성

2) 전체 아키텍처 흐름

현재(v3/v4 공통) 기준선

1. **Trigger**: session이 `StartedAt/FinishedAt`를 기록 (*No-trigger*)
2. **Observe**: curl pod로 `/metrics` 스크랩 후 parse/canonicalize (*External observe*)
3. **Evaluate**: 엔진이 스냅샷을 기반으로 spec 평가/판정 후 summary 생성
4. **Write(Artifacts)**: outPath가 있을 때만 JSON atomic write (v4는 충돌 처리 강화)

3) 구성요소

Trigger 계층

- `NoTriggerBoundary`: `Begin/End=time.Now()` 기반 경계 생성 (v3/v4 현재)
- (추가 예정) `AnnotationTrigger` / `ConditionTrigger`: 시스템 신호 기반 경계 생성 (v4.1~v4.2)

Observe 계층

- `HTTPMetricsObserver`: `/metrics`를 스크랩해 `Snapshot{Metrics map[string]float64}` 생성 (v3/v4 현재)
- (선택) `InternalObserver`: 테스트 내부에서 스냅샷 구성(원칙상 프로덕션 코드 침투 금지)

Evaluate 계층

- `Evaluator`: spec(입력키/계산/판정) 기반으로 결과 계산 후 summary 생성
- 목표: **Observe/Write와 분리**하여 정책 혼재를 줄임(단, v4.x에선 shim으로 점진 적용 가능)

Write 계층(산출물)

- `Writer`: `ArtifactsDir` 비어있으면 noop, 아니면 atomic write
- v4: `NextSummaryPath` 등으로 파일 경로 충돌을 안전하게 처리

4) 공식 지원 조합

현재 지원(확정)

- **[M1] No-trigger + External observe + Evaluate**
  - v3/v4가 여기에 해당
  - 가장 단순하고, "경계가 있으므로" 외부 관측이라도 SLI 판정에 충분히 의미 있음

v4.1~v4.2에서 추가 지원(목표)

- **[M2] Trigger-based + External observe + Evaluate**
  - annotation/condition/status 등으로 경계 생성
  - 관측은 그대로 `/metrics` 기반

- e2e/CI에서 재현성과 “측정 구간의 귀속”이 명확해짐

내부 관측(선택/유보)

- **[M3] Trigger-based + Internal observe + Evaluate**
  - 프로덕션 비침투 원칙 때문에 “테스트 내부에서만” 허용 가능
  - 당장은 v4.x 범위에선 “실험/옵션”으로 두고, 필요성이 명확해지면 승격

5) 업데이트 계획 (v4.1 → v4.2 → v5 조건)

v4.1: Trigger 레이어 도입(최소 기능) + 호환 유지

목표

- `schemaVersion="slo.v3"` 유지
- 기존 M1 경로 유지하면서, M2를 “옵션”으로 추가

작업 항목

- Trigger 인터페이스(또는 개념) 도입: `NoTriggerBoundary` / `AnnotationTrigger`(최소)
- config에 `mode.trigger`를 실제로 의미 있게 채우기 (예: `"none"|"annotation"`)
- v4의 `NextSummaryPath`/충돌 처리 규칙은 유지

산출물/계약

- 기존 v3/v4 결과 파일 네이밍/필드 최대한 동일
- trigger 기반 모드에서 `RunConfig.Tags/TriggerInfo`에 근거 정보 남김

v4.2: EndPolicy 확장(Condition/MetricDelta)로 Trigger 정교화

목표

- “끝점” 결정 방식을 정책화해서 테스트 스위트가 선택 가능하게 함

예시 정책(이미 언급한 축)

- **EndPolicy=StatusCondition**
  - 특정 리소스 조건 True/Ready 도달 시각을 end로 삼음
- **EndPolicy=MetricDelta**
  - 특정 메트릭이 변화(증가/안정화)한 시각을 end로 삼음

작업 항목

- Trigger 구현에 EndPolicy 플러그:
  - condition watcher(테스트/하네스 레벨) 또는
  - metric-based end detector(관측 루프) 추가
- "경계 시각의 출처"를 summary에 남길지 결정(최소는 tags/triggerInfo로)

산출물/계약

- v4.1과 동일하게 스키마는 유지, `config.format="v4"` 등으로 경로 표기

v5로 올리는 조건(명확한 기준)

아래 중 하나라도 해당되면 v5(major)로 격상 권장:

- `pkg/slo`의 **public API가 깨짐** (예: `engine.Execute` 제거/대체 강제)
- 에러/워닝 정책(B1~B5 등)이 **호환 불가 수준으로 변경**
- summary 스키마가 **깨지는 변경**(필드/의미 변경, schemaVersion 변경)

즉, **v4.x는 "호환 유지 + 모드 추가"**,

- *v5는 "API/계약/스키마의 본격 정리"**로 역할 분담.

6) 구성/경로 정책 (Artifacts, format, naming)

- 파일 네이밍: 기존 규칙 유지

  `sli-summary.v3.<run_id>.<test_case>.json` + sanitize
- v4 충돌 처리: `NextSummaryPath` 방식 유지(중복 실행/재시도 안전)
- `ArtifactsDir==""`이면 write는 noop (계산은 가능)
- `config.format`:
  - v3: 빈값 또는 `"v3"`
  - v4: `"v4"`
- `mode.trigger`:
  - v4.1부터 `"none"|"annotation"|...`를 일관되게 채우는 것을 목표

7) 오류/이벤트 기록 정책

- Trigger/Observe 실패를 **테스트 실패로 볼지**는 "정책 옵션"으로 고정
  - 기본: BestEffort(=warnings로 강등) 유지

- 단, “파일 write 실패”의 취급은 명확히 통일 권장
  - v4.x: 기존 계약 유지(현재처럼 일부 경로에서 무시되는 문제는 문서화)
  - v5: `EvaluateResult{Err,Warnings}`로 정책 단일화(권장)

8) 테스트 계획

- 단위 테스트
  - Trigger: Begin/End/Finalize가 경계를 올바르게 생성하는지
  - EndPolicy: Condition/MetricDelta가 종료 시점을 안정적으로 결정하는지(타임아웃/재시도 포함)
- e2e 테스트
  - M1(기존) 회귀 보장
  - M2(신규) 모드별 1개씩 “smoke” 케이스 추가
- 아티팩트 검증
  - `config.format`, `mode.trigger`, tags가 의도대로 채워지는지
  - 경계 시각이 summary에 반영되는지(또는 triggerInfo로 추적 가능한지)

9) 운영 체크리스트

- [ ] 같은 테스트를 반복 실행해도 아티팩트 충돌이 없는가(v4 `NextSummaryPath`)
- [ ] 트리거 기반 모드에서 타임아웃/실패 시 warnings로 잘 내려가는가
- [ ] CI에서 `ARTIFACTS_DIR` 유무에 따라 write/noop이 의도대로 동작하는가
- [ ] `mode.trigger`/`format` 값으로 결과를 분류/집계할 수 있는가

10) 한계 및 로드맵

한계

- Trigger-based는 클러스터 관측(조건/메트릭)을 필요로 하므로 지연/일관성 이슈가 생김
- External observe는 scrape 지연에 따라 “경계 시각”과 “관측 시각”의 차이가 발생할 수 있음

로드맵

- v4.1: Trigger 도입(최소)
- v4.2: EndPolicy 확장으로 실사용성 확보
- v5(조건 충족 시): Evaluate/Observe/Write 계약 정리 + public API 정돈 + error/warning 정책 통일

11) 빠른 시작(개발자 관점)

- 기본(M1): 기존처럼 AttachV4(또는 v3 Attach) + `SLOLAB_ENABLED=1` + `ARTIFACTS_DIR=/...`
- 신규(M2): `mode.trigger="annotation"`(또는 옵션) + EndPolicy 선택 (Condition/MetricDelta)

12) 부록: "현재(v3/v4)는 무엇인가?" 한 줄 정의

- **현재 v3/v4 = No-trigger + External observe + Evaluate**

  (세션이 경계를 잡고, `/metrics`를 외부에서 스크랩해 엔진이 계산)